MediNova — Unidad 3

Diseño arquitectural del sistema con DDD, UML, CQRS y ADRs

Nombre(s): Carlos Hernan Garza Torres Oscar Denilson García Vélez Edwin Benjamín Ramírez Nava	
Matrícula(s): 1986937 1994273 1956899	
Nombre del curso: componentes de lógica de Negocio	
Nombre del profesor: Miguel Ángel Gómez Marroquin	
Módulo / Actividad: Unidad 3 · Caso 01 MediNova:	12/10/2026

Bibliografía / fuentes: Caso 01 MediNova proporcionado para la actividad.

1. Alcance y continuidad con Unidad 2

El diseño se limita al proceso principal de gestión de citas, llegadas/tardanzas, no-shows y resultado de consulta. No se convierten en reglas definitivas las políticas que Unidad 2 marca como pendientes: sobrecupo, reincidencia de no-shows, permisos formales, evidencia obligatoria de cancelaciones, procedimiento definitivo de autorizaciones e indicadores.

Los casos seleccionados aparecen en el BRD y en el BPMN entregados. El PDF denominado UserStories no expone identificadores de historias de usuario independientes; por ello la trazabilidad usa los identificadores BR, FR, BRULE y CA que sí están definidos en el BRD, además del BPMN.

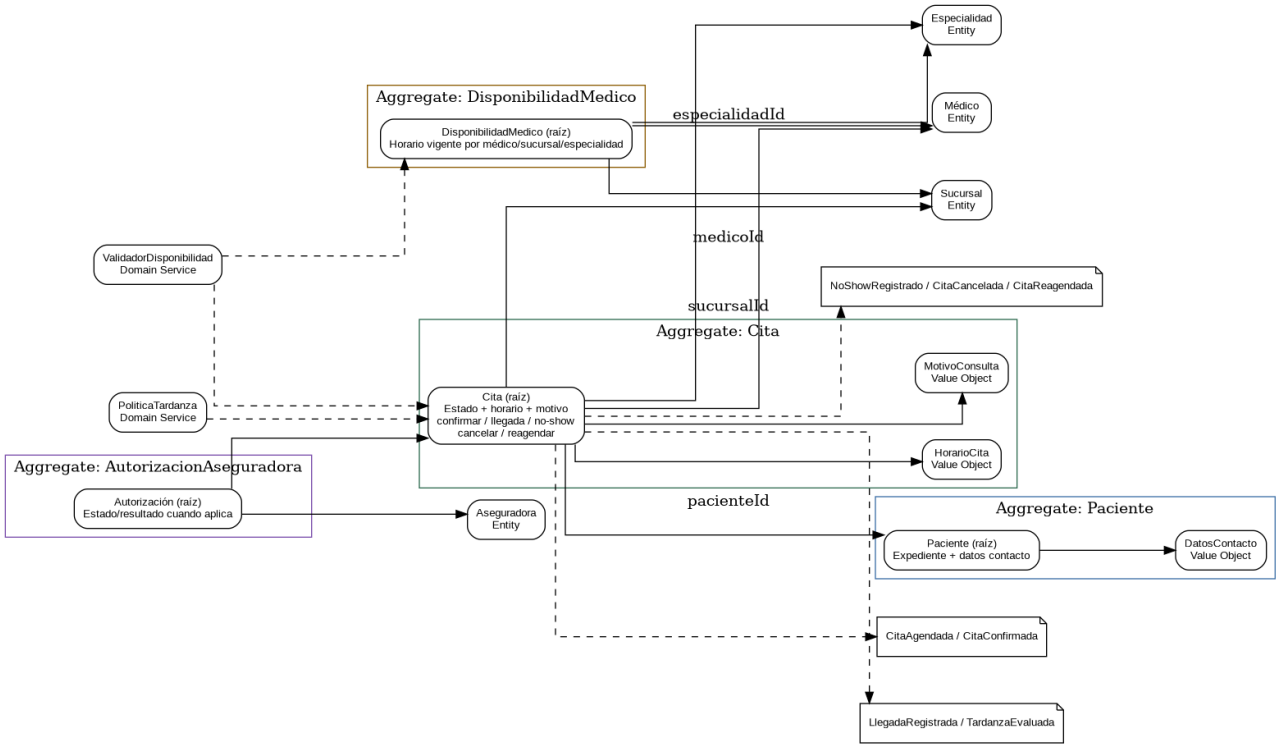
ID	Caso de uso	Actor	Reglas / evidencia	Resultado
CU-01	Agendar cita	Recepcionista	BR-01/02/03; FR-01 a FR-04; CA-01/02	Cita registrada y confirmada si no hay conflicto.
CU-02	Registrar llegada y resolver tardanza	Recepcionista + Médico	BR-05; BRULE-03/04; FR-06/07; CA-06	Llegada registrada y decisión médica o reagenda.
CU-03	Registrar no-show	Recepcionista	BR-04; BRULE-05; FR-08; CA-03	No-show asociado a cita y paciente.
CU-04	Registrar resultado de consulta	Médico	BR-07; FR-12/13; CA-05	Resultado guardado y disponible para historial.

2. Selección del patrón de lógica de negocio

Criterio	Transaction Script	Table Module	Domain Model	Evidencia MediNova
Complejidad de reglas	Media-baja	Media	Alta / adecuada	Disponibilidad, tardanza, no-show, autorización.
Cambios esperados	Cambios en scripts	Cambios por módulo	Reglas encapsuladas	Horarios cambian semanalmente; políticas pendientes.
Interacción entre conceptos	Procedural	Centrada en datos	Entidades/agregados/servicios	Cita interactúa con paciente, médico, disponibilidad.
Estados e invariantes	Difíciles de centralizar	Limitados	Naturales en el dominio	Estados de cita y prevención de duplicados.
Costo	Bajo inicialmente	Medio	Mayor modelado	Se acepta el costo por trazabilidad y reglas.

Decisión: Domain Model. Es el patrón principal porque el caso no es un CRUD simple: hay estados, invariantes y decisiones del negocio que deben permanecer cerca de los conceptos del dominio. El costo aceptado es mayor número de clases, interfaces y pruebas.

3. Mapa de dominio



Aggregates: Cita (raíz principal del proceso), Paciente, DisponibilidadMedico y AutorizacionAseguradora. Value Objects: HorarioCita, MotivoConsulta, DatosContacto y Expediente. Domain Services: ValidadorDisponibilidad y PoliticaTardanza. Domain Events: CitaAgendada, LlegadaRegistrada, TardanzaEvaluada, NoShowRegistrado, CitaCancelada, CitaReagendada, ResultadoConsultaRegistrado y AutorizacionRegistrada.

4. Service Layer

Servicio	Método	Coordinación	Reglas delegadas
AgendarCitaService	agendar(command)	Carga paciente/disponibilidad, valida conflicto, crea cita y confirma UoW.	Disponibilidad, estados de Cita, autorización.
RegistrarLlegadaService	registrar(command)	Carga Cita, registra llegada, persiste y solicita decisión médica cuando aplica.	Regla de 15 min y estados de Cita.
RegistrarNoShowService	registrar(command)	Carga Cita, registra no-show y confirma transacción.	Invariante de asociación no-show/cita.
RegistrarResultadoConsultaService	registrar(command)	Carga Cita, registra resultado y guarda.	Comportamiento de resultado/seguimiento.
ConsultarDisponibilidadHandler	handle(query)	Consulta read model/repositorio.	Ninguna regla de escritura.
ConsultarHistorialPacienteHandler	handle(query)	Obtiene historial para lectura.	Ninguna regla de escritura.

5. Repository y Unit of Work

Abstracción	Métodos clave	Responsabilidad
PacienteRepository	buscarPorTelefono, buscarPorExpediente, guardar	Persistir/recuperar paciente.
CitaRepository	buscarPorId, existeConflicto, listarPorPaciente, guardar	Persistir citas y verificar conflictos.
DisponibilidadRepository	obtenerVigentePorMedico, guardar	Horarios vigentes.
AutorizacionRepository	buscarPorCita, guardar	Resultado de autorización.
ResultadoConsultaRepository	buscarPorCita, guardar	Resultados médicos.
UnitOfWork	begin, commit, rollback	Delimitar la transacción de escritura.

La aplicación depende de interfaces; SQL/ORM queda en infraestructura. Para agendar, la operación se confirma completa o se revierte. La protección contra duplicados debe tener una garantía transaccional de infraestructura además de la validación de dominio.

6. CQRS básico

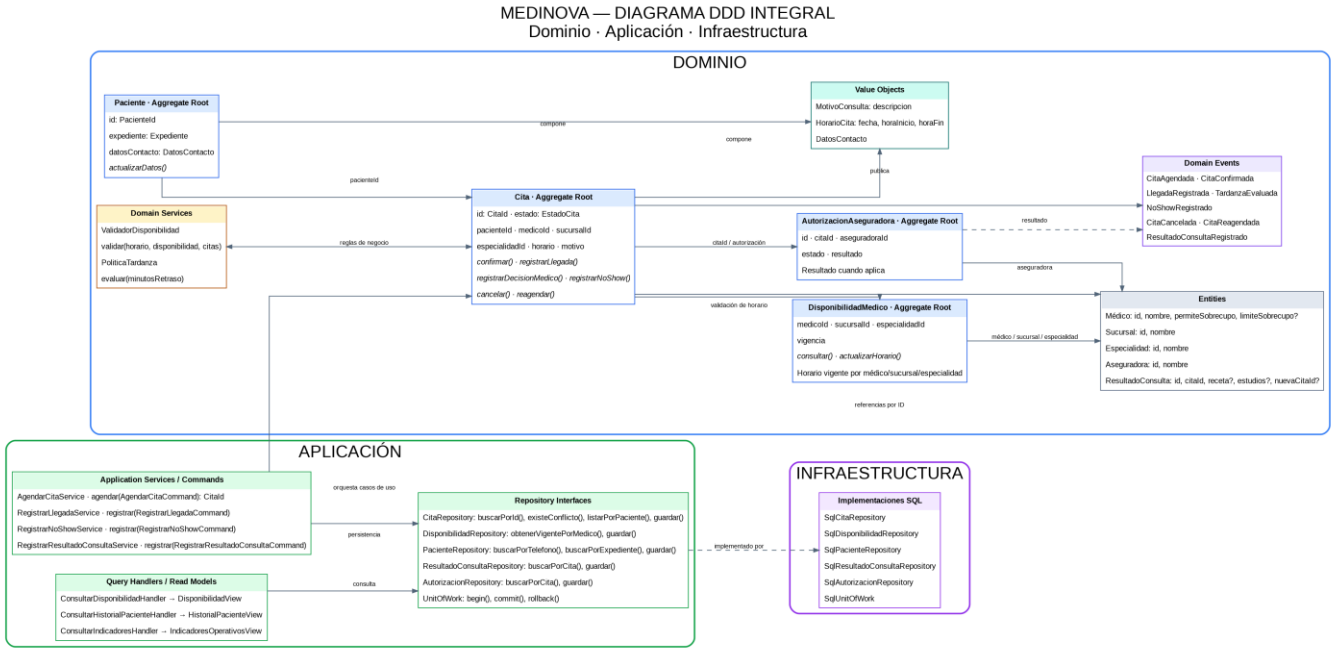
Tipo	Operación	Handler	Salida
Command	AgendarCitaCommand	AgendarCitaService	CitaId
Command	RegistrarLlegadaCommand	RegistrarLlegadaService	ResultadoLlegada
Command	RegistrarNoShowCommand	RegistrarNoShowService	Confirmación
Command	RegistrarResultadoConsultaCommand	RegistrarResultadoConsultaService	Confirmación
Query	ConsultarDisponibilidadQuery	ConsultarDisponibilidadHandler	DisponibilidadView
Query	ConsultarHistorialPacienteQuery	ConsultarHistorialPacienteHandler	HistorialPacienteView
Query	ConsultarIndicadoresQuery	ConsultarIndicadoresHandler	IndicadoresOperativosView

Se aplica CQRS de forma ligera: no se requieren bases de datos separadas ni event sourcing. Los modelos de lectura no se reutilizan automáticamente como modelos de escritura.

7. Revisión SOLID — hallazgos

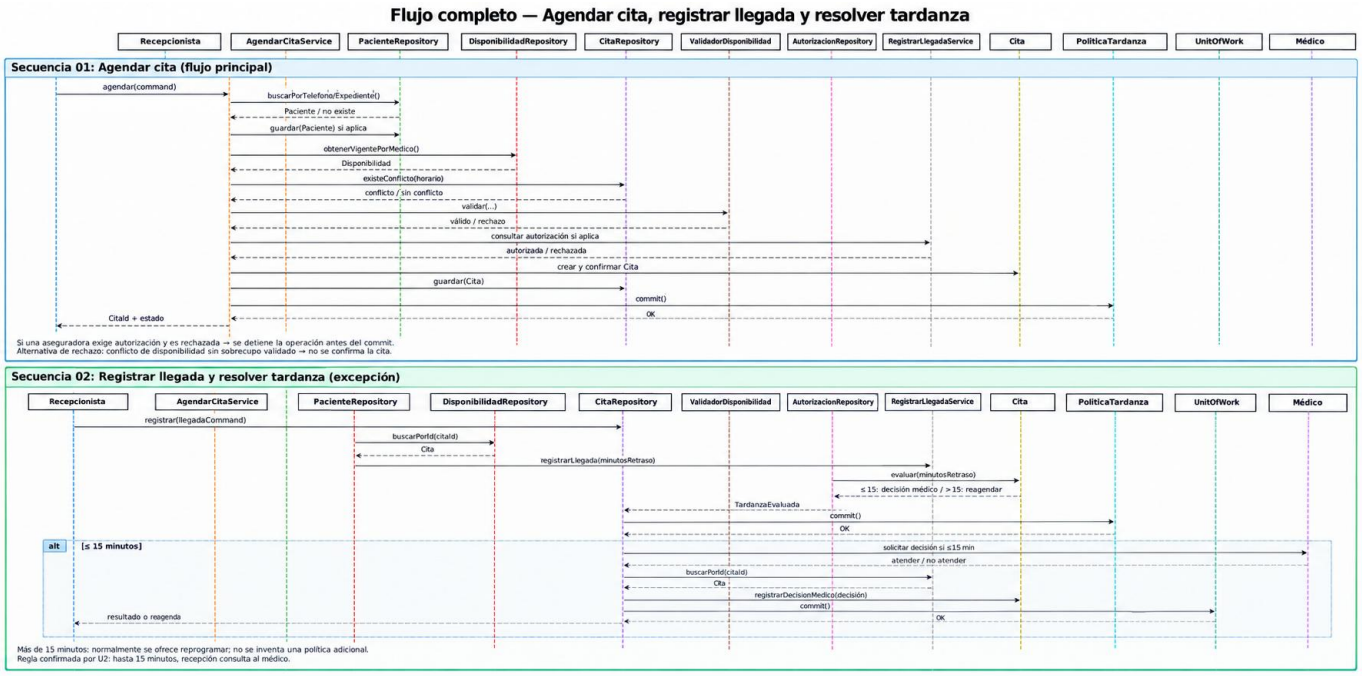
Principio	Hallazgo	Cambio / decisión
S	Cita concentra ciclo de vida; servicios coordinan.	Se evita un Application Service con toda la lógica.
O	Las políticas pendientes no deben generar condicionales rígidos.	Se dejan puntos de configuración/validación.
L	Repositorios SQL deben poder sustituir implementaciones.	Interfaces de dominio/aplicación.
I	Una interfaz única de persistencia sería demasiado grande.	Repositorios separados por necesidad.
D	Dominio/aplicación no deben depender de SQL/ORM.	Infraestructura implementa interfaces.
Consistencia adicional	La prevención de duplicados cruza agregados.	Se usa Unit of Work + garantía transaccional de infraestructura.

8. Diagrama de clases DDD



El diagrama diferencia visualmente Dominio, Aplicación e Infraestructura. Las interfaces Repository, Unit of Work, Application Services, Commands/Queries, agregados, servicios de dominio y eventos están representados.

9. Diagramas de secuencia



10. ADRs

ADR	Decisión	Estado
ADR-001	Domain Model como patrón principal.	Aceptada
ADR-002	Cita como Aggregate Root + UoW para consistencia.	Aceptada
ADR-003	CQRS básico y Application Services delgados.	Aceptada
ADR-004	No inventar políticas pendientes.	Aceptada

ADR-001 — Selección del patrón de lógica de negocio: Domain Model

- Estado: Aceptada
- Fecha: 04/10/2026
- Contexto: MediNova necesita gestionar citas, disponibilidad, tardanzas, no-shows, cancelaciones/reagendas, autorizaciones y resultados de consulta. El caso presenta reglas que interactúan entre conceptos y estados que cambian durante el ciclo de vida de una cita.

Evidencia de Unidad 2

- BR-01 / FR-03 / CA-01: una cita no debe confirmarse si existe conflicto de disponibilidad, salvo sobrecupo validado.
- BR-05 / FR-07 / CA-06: la tardanza de hasta 15 minutos requiere decisión del médico.
- BR-04 / FR-08 / CA-03: el no-show debe quedar asociado a la cita y al paciente.
- BR-07 / FR-12 / FR-13 / CA-05: resultado e historial de la consulta deben conservarse y consultarse.

Alternativas

| Alternativa | Ventajas | Riesgos / consecuencias |

| Transaction Script | Simple y directo para un flujo pequeño. | Las reglas de estado, disponibilidad y excepciones terminarían concentradas en servicios/procedimientos; mayor riesgo de duplicación y servicios extensos. |

| Table Module | Adecuado para reglas principalmente orientadas a registros/tablas. | El caso tiene comportamiento y ciclo de vida que atraviesa paciente, cita, médico y disponibilidad. |

| Domain Model | Encapsula reglas, estados, invariantes y eventos cerca de los conceptos del negocio. | Mayor esfuerzo de modelado, interfaces de persistencia y pruebas de dominio. |

Decisión

Adoptar Domain Model como patrón principal. La entidad `Cita` será responsable de su ciclo de vida y reglas propias; servicios de dominio manejarán comportamientos que cruzan más de una entidad, como la validación de disponibilidad.

No se inventarán políticas pendientes (sobrecupo, reincidencia de no-shows, permisos definitivos). Se representarán como puntos de validación/configuración hasta que el stakeholder las apruebe.

Consecuencias

Positivas: reglas más localizadas, menor acoplamiento con persistencia, mejor trazabilidad y facilidad para emitir eventos de dominio.

Costos: más clases, interfaces y necesidad de definir límites de agregados y transacciones.

Trazabilidad

BR-01, BR-04, BR-05, BR-07; FR-03, FR-07, FR-08, FR-12, FR-13; CA-01, CA-03, CA-05, CA-06.

ADR-002 — `Cita` como Aggregate Root y consistencia transaccional

- Estado: Aceptada
- Fecha: 04/10/2026
- Contexto: La cita concentra el ciclo de vida que el negocio necesita controlar: confirmación, llegada, tardanza, no-show, cancelación y reagenda. Además, confirmar una cita requiere comprobar disponibilidad para evitar duplicidades.

Alternativas

1. Hacer `Paciente` la raíz y anidar todas las citas.

2. Modelar cada cambio como una operación independiente sin Aggregate Root.

3. Hacer `Cita` Aggregate Root y mantener `Paciente`, `DisponibilidadMedico` y `AutorizacionAseguradora` como agregados separados.

Decisión

La Cita será Aggregate Root. Las referencias hacia paciente, médico, sucursal y especialidad se manejarán por identidad. `Cita` protege sus transiciones de estado; la validación de conflicto se coordina mediante `ValidadorDisponibilidad` y `CitaRepository`.

El `UnitOfWork` define el límite de la operación de escritura. Para agendar: creación/actualización de paciente cuando corresponda, registro de cita y resultado de autorización aplicable se confirman juntos; ante error se realiza rollback.

La garantía contra duplicados requiere una protección de infraestructura (por ejemplo, restricción/índice transaccional) además de la validación de dominio, porque es una invariante que cruza agregados.

Consecuencias

- Se mantiene pequeño el límite de consistencia de `Cita`.
- Se evita cargar un agregado gigante que incluya todo el historial del paciente.
- El repositorio puede implementar bloqueo/validación transaccional sin contaminar el dominio con SQL.
- La política de sobrecupo permanece condicionada a una regla validada; no se fija un límite no especificado.

Trazabilidad

BR-01, BR-02, BR-03, BR-04, BR-05; FR-02, FR-03, FR-04, FR-05, FR-06, FR-07, FR-08, FR-09; CA-01, CA-02, CA-03, CA-04, CA-06.

ADR-003 — CQRS básico y Application Services delgados

- Estado: Aceptada
- Fecha: 04/10/2026
- Contexto: MediNova necesita operaciones que cambian estado y consultas de disponibilidad, historial e indicadores. El objetivo es separar claramente escritura y lectura sin introducir bases de datos separadas ni complejidad innecesaria.

Decisión

Aplicar CQRS básico:

Commands

- `AgendarCitaCommand`
- `RegistrarLlegadaCommand`
- `RegistrarNoShowCommand`
- `RegistrarResultadoConsultaCommand`

Queries

- `ConsultarDisponibilidadQuery`
- `ConsultarHistorialPacienteQuery`
- `ConsultarIndicadoresQuery`

Cada operación tiene un handler/application service. Los servicios de aplicación coordinan: reciben el comando/query, cargan agregados mediante interfaces, invocan comportamiento de dominio, persisten y cierran la transacción. No contienen reglas de negocio que pertenezcan a las entidades o servicios de dominio.

Las consultas pueden devolver `DisponibilidadView`, `HistorialPacienteView` e `IndicadoresOperativosView`, sin reutilizar esos DTOs como modelos de escritura.

Alternativas

- Mantener una única interfaz de servicio para lecturas y escrituras: menos piezas, pero mezcla responsabilidades.
- CQRS completo con bases separadas/event sourcing: innecesario para el alcance académico y para la evidencia disponible.
- CQRS básico: equilibrio adecuado entre claridad y complejidad.

Consecuencias

- Mejor separación de responsabilidades y consultas optimizadas para lectura.
- Más handlers y DTOs que una arquitectura CRUD simple.
- No se introduce consistencia eventual ni infraestructura de mensajería obligatoria.

Trazabilidad

BR-02, BR-07, BR-08; FR-02, FR-13, FR-14; CA-02, CA-05, CA-07.

ADR-004 — No convertir políticas pendientes en reglas de dominio definitivas

- Estado: Aceptada
- Fecha: 04/10/2026
- Contexto: La Unidad 2 declara pendientes la política de sobrecupo, reincidencia de no-shows, permisos, evidencia de cancelaciones, procedimiento exacto de autorizaciones e indicadores.

Decisión

La arquitectura modelará únicamente las reglas confirmadas. Las políticas pendientes se expondrán como configuración, interfaces o puntos de validación, pero no se les asignarán valores arbitrarios.

Ejemplos:

- `permiteSobrecupo` puede existir por médico; el límite concreto queda pendiente.
- La regla de tardanza de 15 minutos sí se implementa porque está documentada en BRULE-03/04.
- La reincidencia de no-shows no genera una sanción automática.
- Los permisos se dejan detrás de autorización externa hasta contar con la matriz aprobada.

Consecuencias

- Menor riesgo de convertir supuestos del AS-IS en políticas oficiales.
- Algunas decisiones de implementación quedan abiertas hasta la validación del stakeholder.
- Se requiere mantener una lista de pendientes de validación.

Trazabilidad

BR04/BR05, BRULE-02, BRULE-07, BRULE-08; NFR-01, NFR-02, NFR-03; OI-01 a OI-08.

11. Matriz de Trazabilidad principal

Necesidad / requisito	Diseño relacionado
BR-01 / FR-03 / CA-01	Cita + ValidadorDisponibilidad + CitaRepository + UoW
BR-02 / FR-02 / CA-02	DisponibilidadMedico + DisponibilidadRepository + Query
BR-04 / FR-05/06/08/09 / CA-03/04	Cita + eventos de dominio + trazabilidad
BR-05 / FR-07/10 / CA-06	PoliticaTardanza + RegistrarLlegadaService
BR-07 / FR-12/13 / CA-05	ResultadoConsulta + historial de paciente
BR-08 / FR-14 / CA-07	ConsultarIndicadoresQuery + read model

12. Bitácora de IA y revisión humana

La IA se utiliza como copiloto para proponer alternativas, no como autoridad de diseño. En esta entrega se revisaron las propuestas contra el BRD, glosario y BPMN y se corrigieron especialmente los supuestos sobre sobrecupo, reincidencia de no-shows, permisos e indicadores.

Iteración	Objetivo	Resultado revisado	Decisión humana
1	Comparar patrones	Domain Model resultó mejor para reglas/estados.	Aceptar Domain Model.
2	Proponer agregados	Cita apareció como	Aceptar Cita como raíz y

		núcleo del ciclo de vida.	separar Paciente/Disponibilidad.
3	Proponer Service Layer	Servicios delgados y repositorios por interfaz.	Mover reglas al dominio.
4	Revisar SOLID/CQRS	Separación de commands, queries y persistencia.	Adoptar CQRS básico y registrar ADRs.

13. Conclusión

La arquitectura propuesta transforma el análisis de Unidad 2 en una solución DDD centrada en el ciclo de vida de la Cita, con agregados delimitados, servicios de aplicación para coordinación, repositorios abstractos, Unit of Work, CQRS básico y eventos de dominio. La decisión principal es Domain Model porque MediNova combina disponibilidad variable, estados de cita, excepciones de tardanza, no-shows, autorizaciones y trazabilidad. Las políticas que el caso no define se mantienen abiertas y trazables, evitando que la arquitectura convierta supuestos en reglas de negocio.

14. conclusiones perosnales

Carlos Hernán Garza Torres (Matrícula: 1986937)

La adopción del patrón *Domain Model* resultó fundamental para abordar y controlar la complejidad inherente al ciclo de vida de la cita médica dentro de MediNova. Definir a la entidad Cita como *Aggregate Root* permitió encapsular de forma efectiva sus transiciones de estado, asegurando que las reglas de negocio asociadas a tardanzas, no-shows y reagendamientos permanezcan estrictamente dentro del dominio en lugar de dispersarse en scripts de aplicación o controladores. Un aprendizaje clave de este proceso fue comprender que mantener los límites de los agregados lo más pequeños y definidos posible previene el acoplamiento innecesario con entidades de gran tamaño (como Paciente), facilitando la mantenibilidad y la emisión de eventos de dominio ante cambios de estado.

Oscar Denilson García Vélez (Matrícula: 1994273)

La implementación de un esquema básico de CQRS aportó una separación clara entre las operaciones de escritura que alteran el estado del sistema y las consultas operativas. La incorporación del patrón *Unit of Work* y la delimitación estricta de repositorios mediante interfaces garantizaron la consistencia transaccional en flujos críticos como la asignación de citas o el registro de resultados, sin necesidad de recurrir a la sobrecomplejidad técnica que implicaría una infraestructura completa de *Event Sourcing*. Esta estrategia permitió optimizar las lecturas a través de modelos de vista dedicados (*Read Models*) y reforzar el principio de inversión de dependencias, asegurando que las herramientas de persistencia de infraestructura permanezcan totalmente desacopladas del núcleo de negocio.

Edwin Benjamín Ramírez Nava (Matrícula: 1956899)

El diseño arquitectónico demostró que el rigor metodológico en la documentación de decisiones es tan crítico como la elección de los patrones de código. A través de la elaboración de los Registros de Decisiones de Arquitectura (ADRs) y la matriz de trazabilidad, fue posible alinear cada componente del diseño con las reglas de negocio (BR), requisitos funcionales (FR) y criterios de aceptación (CA) definidos previamente. La principal aportación de este enfoque radicó en evitar la transformación de supuestos u omisiones del negocio en políticas definitivas de software, garantizando que el sistema proporcione los puntos de extensión necesarios para que las reglas pendientes (como sobrecupos o sanciones por no-shows) se puedan integrar fácilmente sin comprometer la estabilidad del sistema.